

使用 Visual Studio Code，透過 Cloud Functions for Node.js 進行本機開發

來源：<https://codelabs.developers.google.com/codelabs/local-development-with-cloud-functions?hl=zh-tw>

步驟數：8

瞭解如何在本機電腦的 Visual Studio Code 中，為 Node.js 撰寫、部署及偵錯 Cloud Functions。

目錄

1. [總覽](#)
2. [設定和需求](#)
3. [安裝 Node.js 適用的 Functions Framework](#)
4. [在本機建立及測試 HTTP Cloud 函式](#)
5. [從本機電腦偵錯 HTTP 函式](#)
6. [從本機電腦將 HTTP 函式部署至 Google Cloud](#)
7. [清除所用資源](#)
8. [恭喜！](#)

步驟 1 · 約 2 分鐘

1. 總覽

[Google Cloud Functions](#) 是事件導向的無伺服器運算平台，有了 Cloud Functions，您就能專心編寫程式碼，不必擔心資源佈建或資源調度作業，以因應不斷變化的需求。

以 JavaScript 編寫的 Cloud Functions 會在 Google Cloud Platform 的 Node.js 環境中執行。您可以在任何標準 Node.js 執行階段中執行 Cloud 函式，以啟用可攜性和本機測試。

逐步操作說明

在本程式碼研究室中，您將建立 Node.js 適用的 Cloud 函式，回報指定溫度是否可接受或過高。您將在本機上使用 Visual Studio Code 建立、測試及偵錯 Cloud Function。最後，您會將函式部署至 Google Cloud Platform。

課程內容

- Node.js 適用的 Functions Framework。
 - 在本機建立及測試 HTTP Cloud 函式。
 - 從本機電腦偵錯 HTTP 函式。
 - 從本機電腦部署 HTTP 函式。
-

2. 設定和需求

必要條件

- [Cloud SDK](#)
- [Visual Studio Code](#)
- Node.js 8.6.0 以上版本 (如要安裝 Node.js，請[使用 nvm](#)；如要檢查版本，請執行 `node -version`)
- 完成「[我的第一個函式：Node.js 指南](#)」。

費用

雖然本程式碼研究室只需要叫用已部署的 Cloud 函式 1 次，您仍應參閱 [Cloud Functions API 定價資訊](#)，瞭解計費方式。

雖然許多 Google API 都能免費使用，但 Google Cloud Platform (即其產品和 API) 並非免費。您必須具備有效的帳單帳戶，才能使用 Cloud Functions。請注意，部分 Google Cloud Platform (GCP) 產品提供「[一律免費](#)」方案，您必須超出用量才會產生費用。在本程式碼研究室中，每次叫用 Cloud Functions 都會計入免費方案。只要每月總用量不超過限制，就不會產生任何費用。

3. 安裝 Node.js 適用的 Functions Framework

[Node.js 適用的 Functions Framework](#) 是開放原始碼的 FaaS (函式即服務) 架構，可供您編寫可攜式 Node.js 函式，由 Google Cloud Functions 團隊提供。

Functions Framework 可讓您編寫簡易函式，並在許多不同環境中執行，包括：

- Google Cloud Functions
- 您的本機開發機器
- Cloud Run 和 Cloud Run on GKE
- 以 Knative 為基礎的環境

建立新的 node.js 應用程式。



```
npm init
```

接受預設值時，請務必使用 `index.js` 做為應用程式的進入點。

接著安裝 Node.js 適用的 Functions Framework。

```
npm install @google-cloud/functions-framework
```

開啟 `package.json`。確認函式架構列為依附元件，如下例所示。

```
"dependencies": {  
  "@google-cloud/functions-framework": "^1.7.1"  
}
```

Functions Framework 現已安裝完成。您現在可以建立 Cloud Function。

步驟 4 · 約 5 分鐘

4. 在本機建立及測試 HTTP Cloud 函式

建立本機 Cloud 函式

在本節中，您將建立並測試可回應 HTTP 要求的 HTTP 函式。

在 `package.json` 檔案所在的目錄中，建立名為 `index.js` 的新檔案。

新增下列程式碼：

```
exports.validateTemperature = async (req, res) => {
  try {
    if (req.body.temp < 100) {
      res.status(200).send("Temperature OK");
    } else {
      res.status(200).send("Too hot");
    }
  } catch (error) {
    //return an error
    console.log("got error: ", error);
    res.status(500).send(error);
  }
};
```

現在可以測試函式了。

在 Visual Studio Code 中測試函式

從現在開始，本程式碼研究室會使用 Visual Studio Code 內的 [整合式終端機](#)。

在 Visual Studio Code 中開啟終端機視窗。

執行下列指令：

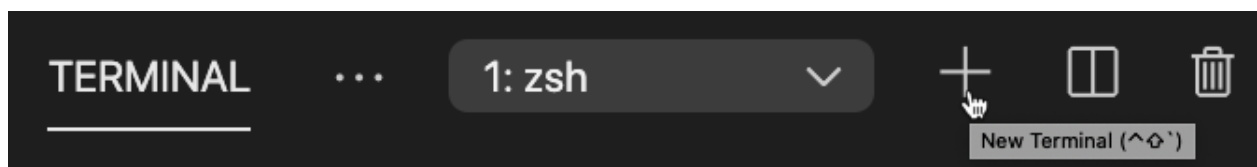
```
node node_modules/@google-cloud/functions-framework --target=validateTemperature
```

這個指令會啟動本機伺服器，當伺服器收到 HTTP 要求時，即可呼叫 `validateTemperature` 函式。

終端機視窗中應該會顯示下列輸出內容：

```
Serving function...
Function: validateTemperature
URL: http://localhost:8080/
```

在 VS Code 中建立第二個終端機視窗，方法是在「終端機」視窗窗格中點選 `New Terminal` 加號圖示。您會在兩個終端機視窗之間切換：第一個視窗用於提供函式，第二個視窗則用於使用 `curl` 呼叫函式。



您可以使用下拉式選單切換終端機視窗。如果終端機視窗目前正在提供函式，下拉式清單會將其稱為 `node`。否則，請參閱 `zsh` (或您使用的殼層)。

在第二個終端機視窗中，執行下列指令，將溫度酬載 50 傳送至提供 `validateTemperature` 函式的本機伺服器。

```
curl -X POST http://localhost:8080 -H "Content-Type:application/json" -d
'{"temp": "50"}'
```

您應該會收到下列 Cloud Function 回應：

```
Temperature OK
```

在第二個終端機視窗中，傳送「過高」的溫度有效負載，再次測試函式，如下所示：

```
curl -X POST http://localhost:8080 -H "Content-Type:application/json" -d
'{"temp": "120"}'
```

您應該會收到下列 Cloud Function 回應：

```
Too hot
```

最後，使用缺少酬載的呼叫來測試函式。

```
curl -X POST http://localhost:8080
```

您應該會收到下列 Cloud Function 回應：

```
Too hot
```

如果未提供溫度，函式最好不要傳回「太熱」。您發現程式碼有錯誤。

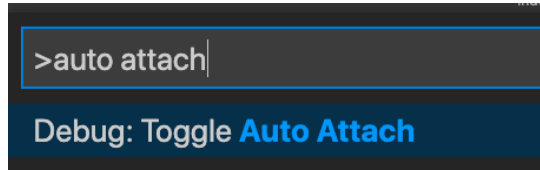
請務必在提供函式的第一個終端機視窗中按下 `Ctrl + C`，停止執行函式。

步驟 5 · 約 10 分鐘

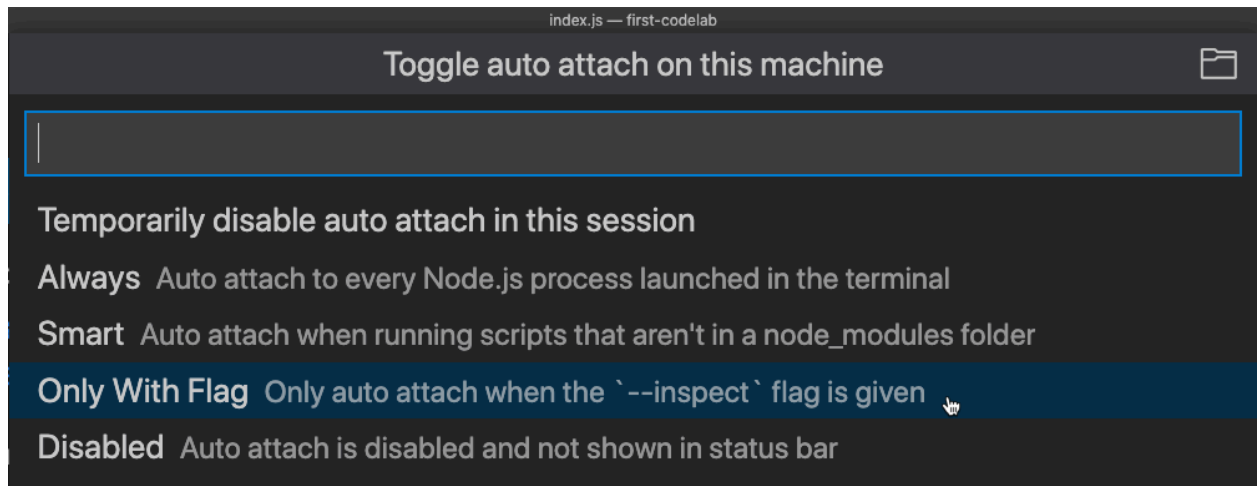
5. 從本機電腦偵錯 HTTP 函式

在 Visual Studio Code 中開啟指令區塊面板。如果是使用 Mac，請改用 `Cmd + Shift + P`。如果你使用 Windows，請使用 `Ctrl + Shift + P`。

在指令區塊面板中輸入 `auto attach`，然後從清單中選取第一個項目。

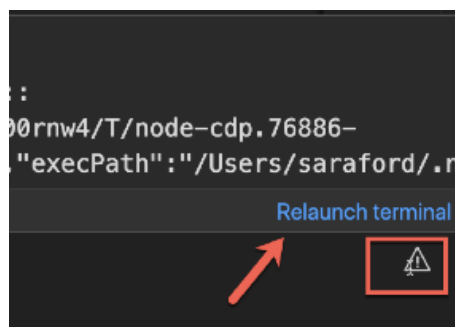


在本程式碼研究室中，請選擇 `Only With Flag`，如下圖所示：



現在請重新載入您在 VS Code 中用來提供函式的終端機視窗，方法是將滑鼠游標懸停在最右側顯示的警告圖示上。

按一下「`Relaunch Terminal`」。



在重新載入的終端機視窗中，重新執行 Functions Framework，使用下列指令提供函式：

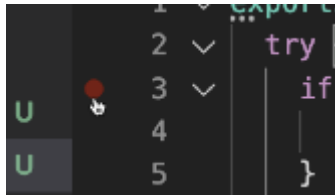
```
node --inspect node_modules/.bin/functions-framework --target=validateTemperature
```

其中 `--inspect` 旗標會告知 Node.js 監聽偵錯用戶端。詳情請參閱 [Node 偵錯說明文件](#)。

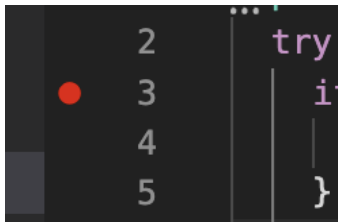
請注意，您使用的是 `node_modules/.bin/functions-framework`，而非 `node_modules/@google-cloud/functions-framework`。如要使用檢查模式，您必須使用 `/node_modules/.bin` 中自動[符號連結的可執行檔](#)。

這次您應該會在 VS Code 中看到橘色的狀態列，表示偵錯工具已附加。

在行號左側的邊界內按一下，在第 3 行設定中斷點。



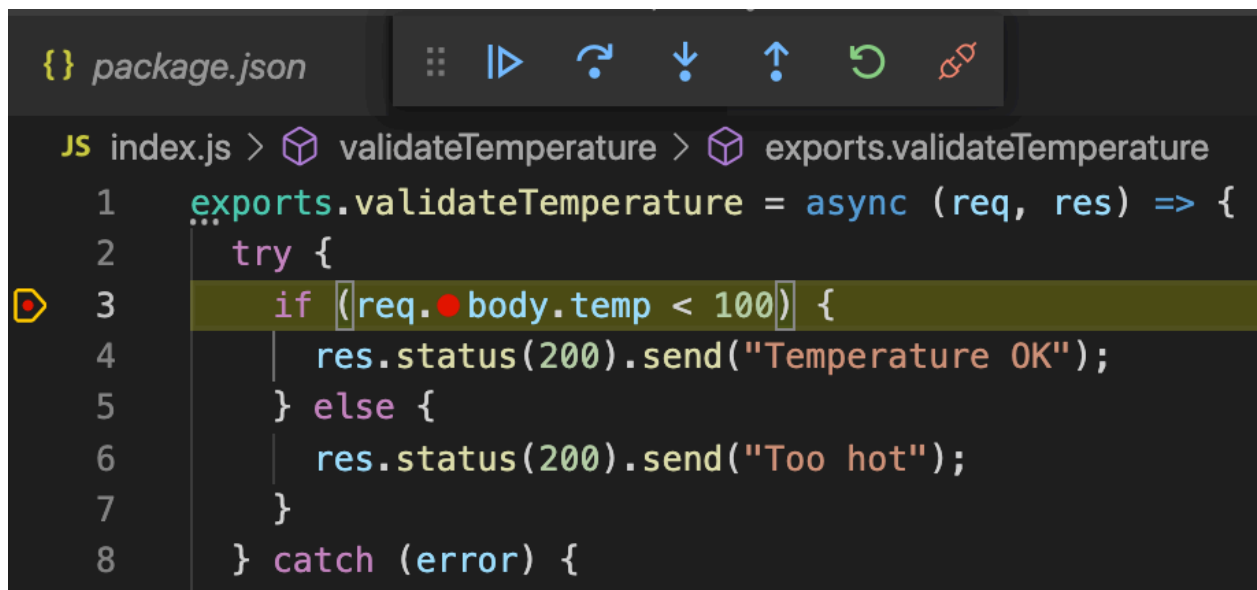
中斷點圖示應會亮起鮮紅色，表示偵錯工具可存取這行程式碼。



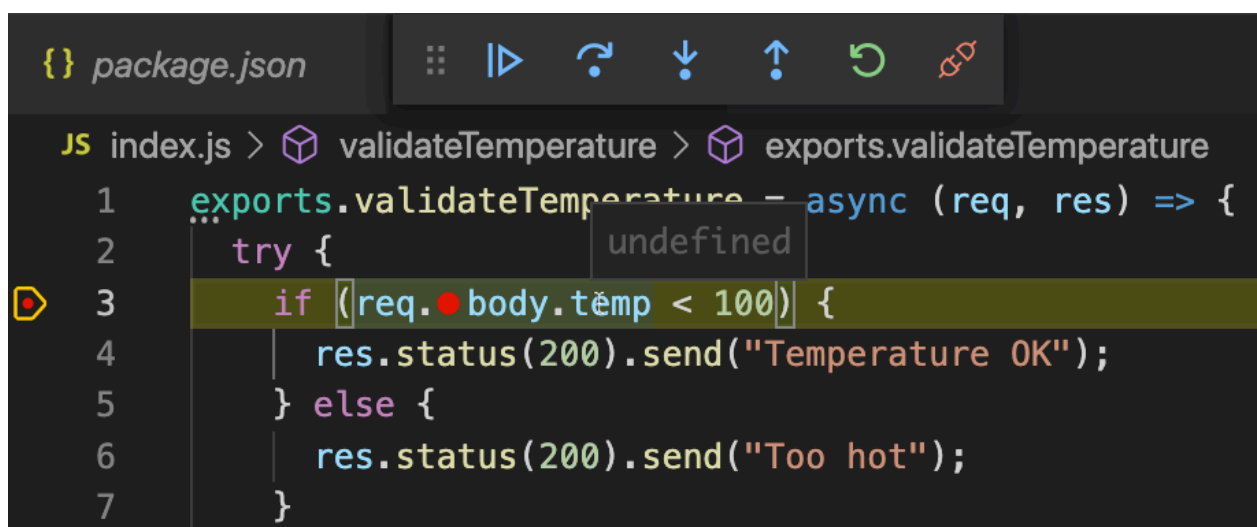
在第二個終端機視窗中，執行下列 curl 指令，觸發中斷點。

```
curl -X POST http://localhost:8080
```

第 3 行會以黃色醒目顯示。這項醒目顯示效果表示，這行是偵錯工具目前評估的陳述式。



將滑鼠游標懸停在暫時變數上，確認其內容為 `undefined`，因為要求未提供溫度酬載。



按一下「逐步執行」圖示，執行下一個陳述式。

您會看到目前的陳述式跳至 if 陳述式的 else 部分。

```
JS index.js > validateTemperature > exports.validateTemperature
1  exports.validateTemperature = async (req, res) => {
2    try {
3      if (req.body.temp < 100) {
4        res.status(200).send("Temperature OK");
5      } else {
6        res.status(200).send("Too hot");
7      }
8    } catch (error) {
9      //return an error
```

在本範例中，您可以假設規格要求所有要求都必須傳送溫度讀數。如果溫度讀數未提供，函式應擲回例外狀況。

按一下「中斷連線」按鈕，即可中斷偵錯工具連線。

```
{ } package.json
JS index.js > validateTemperature > exports.validateTemperature
1  exports.validateTemperature = async (req, res) => {
2    try {
```

在第一個終端機視窗中，按下 `Ctrl + C` 停止執行函式。

更新函式，新增 if 陳述式，在溫度未定義時擲回例外狀況，如下所示：

```
exports.validateTemperature = async (req, res) => {

  try {

    // add this if statement below line #2
    if (!req.body.temp) {
      throw "Temperature is undefined";
    }

    ...
```

在第一個終端機視窗中，再次執行下列指令(不含 `-inspect` 旗標)，重新啟動 Cloud Functions，避免附加偵錯工具。

```
node node_modules/@google-cloud/functions-framework --target=validateTemperature
```

在第二個終端機視窗中執行下列指令，確認是否擲回例外狀況：

```
curl -X POST http://localhost:8080
```

要求應會傳回下列輸出內容：

```
Temperature is undefined
```

在第一個終端機視窗中，您也會看到函式記錄的錯誤。

```
Serving function...  
Function: validateTemperature  
URL: http://localhost:8080/  
got error:  Temperature is undefined
```

現在，您可以在第一個終端機視窗中按下 Ctrl + C，停止執行函式。

步驟 6 · 約 5 分鐘

6. 從本機電腦將 HTTP 函式部署至 Google Cloud

您已在本機電腦上建立、測試及偵錯 Cloud Function，現在可以將其部署至 Google Cloud。

執行下列指令，確認您在本機使用的是步驟 2 中建立的專案：

```
gcloud config get-value project
```

如果您在步驟 2 中指定的專案不是有效設定，請執行下列指令：

```
gcloud config set project <project-name-created-step-2>
```

在任何終端機視窗中執行下列指令：

```
gcloud functions deploy validateTemperature --trigger-http --runtime nodejs12 --allow-unauthenticated
```

參數說明如下：

- `deploy validateTemperature`：用於部署名為 `validateTemperature` 的 Cloud Functions，且進入點名為 `validateTemperature` 的 gcloud 子指令
- `--trigger-http` - 觸發事件類型
- `--runtime nodejs12` - 這個函式的目標執行階段
- `--allow-unauthenticated`：允許公開存取，可呼叫函式

系統會提示您啟用 Cloud Functions API。輸入 `y` 啟用 API。

```
API [cloudfunctions.googleapis.com] not enabled on project
[1057316433766]. Would you like to enable and retry (this will take a
few minutes)? (y/N)? y
```

部署完成後，輸出內容會顯示以下資訊：

```
Deploying function (may take a while - up to 2 minutes)...done.
availableMemoryMb: 256
buildId: <your-build-id>
entryPoint: validateTemperature
httpsTrigger:
  url: https://<your-region-and-project>.cloudfunctions.net/validateTemperature
...
```

在終端機視窗中，使用 `curl` 呼叫這個公開端點。

```
curl -X POST https://<your-region-and-project>.cloudfunctions.net/validateTemperature
-H "Content-Type:application/json" -d '{"temp": "50"}'
```

並確認適當的回應，確認雲端函式已成功部署。

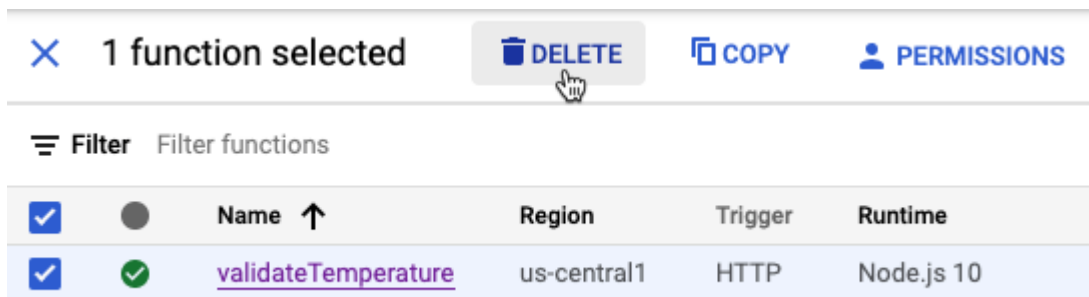
Temperature OK

7. 清除所用資源

為避免產生非預期費用 (例如，Cloud 函式遭非預期呼叫的次數超過[免費層級的每月 Cloud 函式呼叫配額](#))，您可以刪除 Cloud 函式，或刪除您在步驟 2 中建立的專案。

如要刪除 Cloud Function，請前往 Cloud Function Cloud 控制台：<https://console.cloud.google.com/functions/>。請確認目前選取的專案是您在步驟 2 中建立的專案。

選取您在步驟 6 部署的 `validateTemperature` 函式。然後按一下「刪除」。



如要刪除整個專案，請前往 <https://console.cloud.google.com/cloud-resource-manager>，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。

步驟 8 · 約 1 分鐘

8. 恭喜！

恭喜您完成本程式碼研究室。您可以進一步瞭解 [Cloud Functions 如何支援 Node.js 執行階段](#)，以及 [Cloud Functions 的本機偵錯功能運作方式](#)。

涵蓋內容

- Node.js 適用的 Functions Framework。
 - 在本機建立及測試 HTTP Cloud 函式。
 - 從本機電腦偵錯 HTTP 函式。
 - 從本機電腦部署 HTTP 函式。
-